C++ Project: Matrix and Linear Operations

Adrien Evo
Rémi Schmitt
Théo Verdelhan

February 2026

1 Topic presentation

The objective of this project is to design and implement a `Matrix` class in C++, allowing the manipulation of real-valued matrices and the execution of the fundamental matrix operations.

The class must:

- Store coefficients in a `std::vector<double>`
- Keep track of dimensions (number of rows and columns)
- Implement the following operations:
  - Matrix addition
  - Matrix × matrix multiplication
  - Matrix × vector multiplication
  - Transpose
  - Determinant (for small square matrices)
- Overload the following operators:
  - `operator+`
  - `operator*`
  - `operator==`
  - `operator()` for accessing coefficients
- Handle errors via:
  - `std::invalid_argument` for incompatible dimensions
  - `std::out_of_range` for out-of-bounds indices

This implementation provides a foundation for matrix computations in numerical linear algebra.

1.1 Implementation challenges

1.1.1 The importance of memory contiguity: the `std::vector<std::vector<double>>`

Intuitively, it may seem tempting to represent a two-dimensional matrix using a "vector of vectors". However, this approach is not efficient for intensive computations.

A vector of vectors is in fact a dynamic array of pointers, where each sub-vector (each row of the matrix) is allocated independently on the heap.

1.1.2 Hardware optimization: preventing "cache misses"

Since main memory (RAM) is much slower than the memory directly on the CPU (cache), when the processor needs a piece of data, it first looks for it in its cache. If it finds it, it is a *cache hit*. If it does not, it is a *cache miss*. The processor is then forced to fetch the data from RAM, which is slower.

To optimize transfers, the CPU never loads a single variable at a time from RAM. It loads entire contiguous blocks of memory.

If the data is contiguous in memory, loading one entry of the matrix will automatically bring the subsequent entries into the cache. With a vector of vectors scattered in memory, the CPU must fetch coefficients from RAM repeatedly, drastically reducing the speed of algorithms such as matrix multiplication.

1.1.3 The solution: linearization into a 1D vector

To guarantee strictly contiguous memory usage, the key challenge was to model the mathematical 2D matrix using a single 1D vector (`std::vector<double>`).

This choice implies a mathematical translation (linearization) of coordinates. To access the element located at row i and column j in a matrix with C columns, the corresponding 1D index is computed by:

$$\text{Index} = i \times C + j$$

2 Mathematical reminders

2.1 General definition of a matrix

A matrix A of dimension $m \times n$ is a set of $m \times n$ real coefficients denoted:

$$A = (a_{ij})_{1 \leq i \leq m,\ 1 \leq j \leq n}$$

where:

- m is the number of rows,
- n is the number of columns,
- a_{ij} is the coefficient at row i and column j.

A matrix can be explicitly represented as:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Matrices are used in linear algebra to represent:

- systems of linear equations,
- linear transformations,
- change of basis,
- structured numerical data.

In this project, we manipulate real-valued matrices stored in memory in vector form.

2.2 Matrix addition

Let A and B be two matrices of dimension $m \times n$. Addition is defined by:

$$(A + B)_{ij} = A_{ij} + B_{ij}$$

Necessary condition: the matrices must have the same dimensions.

2.3 Matrix $\times$ matrix multiplication

Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. The product $C = AB$ is a matrix of dimension $m \times p$ defined by:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

Necessary condition: the number of columns of A must equal the number of rows of B.

2.4 Matrix $\times$ vector multiplication

Let $A \in \mathbb{R}^{m \times n}$ and $x \in \mathbb{R}^n$.

$$(Ax)_i = \sum_{j=1}^{n} A_{ij} x_j$$

Condition: the vector size must match the number of columns of the matrix.

2.5 Transpose

The transpose of a matrix A is defined by:

$$(A^T)_{ij} = A_{ji}$$

If A has dimension $m \times n$, then A^T has dimension $n \times m$.

2.6 Determinant

The determinant is defined only for square matrices.

2.6.1 General case: cofactor expansion

$$\det(A) = \sum_{j=1}^{n} (-1)^{1+j} A_{1j} \det(M_{1j})$$

where M_{1j} is the submatrix obtained by removing the first row and the j-th column.

This method is recursive and only suitable for small matrices (factorial complexity).

3 Non-trivial design choices regarding code organization: separating interface and implementation

Beyond hardware performance, particular care was taken in the software architecture of our class, following C++ design standards. This is illustrated notably by the management of utility functions (*helpers*), especially those required for computing the determinant.

3.1 Free functions and the encapsulation principle

Rather than declaring the determinant utility functions as private methods of the `Matrix` class, we chose to implement them as free (non-member) functions directly in the source file (`.cpp`).

This approach follows the recommendation of C++ expert Scott Meyers: if a function can accomplish its task by only calling the public interface of a class, it should be implemented as a free function rather than as a member method. This increases encapsulation by reducing the total number of functions with direct access to the private internal data.

3.2 Isolation using anonymous namespaces

Since these utility functions are not intended to be exposed to the user of the class, it was necessary to restrict their scope. To do so, we encapsulated them within an anonymous namespace (`namespace { ... }`) inside the `.cpp` file.

In C++, placing an entity in an anonymous namespace gives it internal linkage. These functions then become strictly invisible and inaccessible from other project files. This prevents polluting the global namespace and avoids potential name collisions (contrary to the One Definition Rule) during the linking stage.

3.3 Optimizing compilation dependencies

This design choice also brings advantages if our class were to be integrated into a larger project:

- The header file remains concise. It is not polluted by implementation details and acts as a clear contract for the user.
- Minimization of cascading recompilations: the C++ compilation model implies that modifying a header forces recompilation of all source files that include it. By confining implementation and helper functions to the `.cpp`, any modification (bug fix, determinant optimization) only requires recompiling that isolated source file.

4 Going further: perspectives and evolutions

Although our current implementation meets the requirements of the topic, several improvement paths can be considered to make this matrix class more robust, efficient, and generic.

4.1 Dynamic resizing and memory management

Currently, the matrix size is fixed after instantiation. Allowing the user to dynamically add rows or columns would require modifying the underlying 1D vector structure.

If adding a row at the very end of the matrix is relatively inexpensive (similar to successive `push_back` calls), inserting a column, or inserting a row in the middle, is more complex. It requires shifting a large number of elements in memory to preserve row-major storage (row by row). Furthermore, these insertions can exceed the allocated capacity of the vector, forcing a complete memory reallocation. These resizing methods must therefore be designed carefully to minimize performance degradation.

4.2 Algorithmic optimization: determinant computation

In this project, the determinant computation relies on Laplace's formula (expansion along a row or a column). This method has factorial complexity on the order of $O(n!)$. While this is not a major issue for the small matrices targeted by our topic, the algorithm quickly becomes impractical as the dimension increases.

For a scalable implementation, it would be necessary to replace this approach with Gaussian elimination or LU decomposition. These methods triangularize the matrix, reducing complexity to $O(n^3)$ (the determinant of a triangular matrix being easily computed as the product of its diagonal elements).

4.3 Genericity with templates

Our class is currently restricted to storing floating-point numbers (`double`). A natural C++ evolution would be to turn it into a class template using `template <typename T>`, thus using a `std::vector<T>`.

While the idea is simple, it raises additional challenges when overloading arithmetic operators. For instance, it would be necessary to properly manage operations between matrices of different types (e.g., adding a `Matrix<int>` with a `Matrix<double>`). This would require mastering implicit conversions and using modern tools such as `std::common_type` to deduce the correct return type without losing precision.

4.4 Advanced mathematical functionalities

Finally, enriching the mathematical interface would open the door to more complex (non-linear) computations. Implementing eigenvalue and eigenvector algorithms would allow matrix diagonalization. This property is fundamental: it enables fast computation of matrix powers ($A^k = PD^kP^{-1}$) and, by extension, computation of the matrix exponential (e^A), an essential operation in many areas of physics and engineering.